

Software Self-evolution with SELF-EVOLVE: an Agentic Architecture for Runtime Code Generation

Anonymous Author(s)

Abstract

Traditional self-adaptive systems automatically reconfigure existing components in response to changing requirements, but cannot generate new functionalities at runtime. The software generation capabilities of large language models (LLMs) open the possibility to create entirely new modules at runtime, enabling a form of self-evolution beyond traditional self-adaptation. We present SELF-EVOLVE, an orchestrated pipeline architecture enabling runtime self-evolution through iterative code generation based on automatic test-driven development. Evaluation on HumanEval problems and complex integration tasks demonstrates promising results: 77.8% success rate versus 46.7% for TiCoder, 100% Pass@1 on 10 HumanEval problems, and successful integration on complex tasks. This work establishes a new paradigm in which software systems autonomously extend their capabilities during execution, paving the way toward long-lived, self-improving systems.

1 Introduction

Software systems typically operate with fixed capabilities determined at development time. Adding new functionality requires human developers to write code, test it, and deploy updates—a process taking days to weeks. Automated code generation with LLM agents [4] offers the possibility to speed-up development time, with speed gains of about 21%, based on a controlled experiment at Google [17]. Commercial code generation tools include GitHub Copilot [7], CodeT5 [18], and TiCoder [5], as well as more complex frameworks, such as ClarifyGPT [16], which enables requirements clarification before code generation, or the self-planning approach by Jiang et al. [11], focused on automatic problem decomposition, or multi-agent frameworks like MetaGPT [10] simulating development teams. However, these approaches do not support runtime code generation and integration—they cannot enable software to autonomously extend itself during execution. Such a capability would be transformative for software development, allowing users to experience applications that adapt to their needs in real time, creating personalized versions. Consider the following example. A Netflix user asks: “show me what my neighbors are watching.” The system lacks neighborhood analytics, but rather than returning an error, it autonomously generates this capability—synthesizing the function, validating it through tests, integrating it into its runtime, and transparently delivering results with minimal delays. Once proven reliable through usage, this new capability can become available to all users.

Although Multi-Agent Self-Evolving (MASE) systems have been recently introduced in the literature [6], these are typically oriented to refine and optimise their prompts, memory, and tool-use strategies—e.g., reaching existing services through the model context protocol (MCP)¹—but do not generate functions at runtime. Studies in evolutionary computation [20], e.g., using genetic algorithms, have explored the potential of LLM to generate program

variations that are more human-like and functional, but none of the work is focused on evolving software systems at runtime by generating and integrating novel functionalities. Recent works have explored the possibility of using LLMs to iteratively refine code [8, 12], but runtime evolution has not been addressed. Similarly, pioneering works in self-evolution, such as Dellarocas et al. [3] and, more recently, Weins et al. [19], mainly focused on structural adaptation and reconfiguration. Finally, traditional works on runtime code generation [13] mainly focus on low-level code optimization or patching, rather than functionality extension.

In this paper, we present SELF-EVOLVE, an first architecture designed for self-evolution through runtime code generation. Our orchestrated pipeline coordinates specialized components—dispatcher, test generator, code synthesizer, sandboxed executors, and context memory—with defined interfaces, ensuring isolation between generation and core functionality of the software to evolve. The architecture uses test-driven development (TDD) principles as its core, and employs iterative refinement: automated diagnostic feedback based on test execution guides regeneration until convergence. Our evaluation aims to answer three research questions (RQs):

RQ1: *What are the code generation capabilities of SELF-EVOLVE compared to existing baselines?* This RQ evaluates SELF-EVOLVE in a scenario where the newly generated code does not require integration with other functions, and we compare the quality of the generated code with similar baselines.

RQ2: *What are the self-evolution capabilities enabled by SELF-EVOLVE?* This RQ tests self-evolution in contexts where generated functions need to be integrated.

RQ3: *How do the TDD pipeline contribute to self-evolution capability?* (ablation study) Here we specifically focus on the relevance of the TDD pipeline, which is at the core of our architecture.

Results demonstrate 77.8% success versus 46.7% for baselines ($p=0.50$, $r=0.62$), 100% Pass@1 with TDD versus 90% without, and successful integration with 500+ line codebases. While limited sample sizes prevented statistical significance, large effect sizes suggest practical importance.

Data Availability. We release our implementation at <https://anonymous.4open.science/r/self-evolving-systems-FEC4>, including all components, evaluation scripts, and experimental configurations.

2 SELF-EVOLVE Architecture

SELF-EVOLVE implements an orchestrated pipeline of specialized components for autonomous runtime code generation. Three technical solutions enable runtime updates: persistent Knowledge Base architecture, dynamic module integration, process isolation. In the standard behavior, a user requests functionality in natural language (using OpenAI function-calling format²), this is already available in a persistent Knowledge Base, and it is thus executed. The evolution process begins when users request an unavailable functionality—the

¹<https://modelcontextprotocol.io/>

²<https://platform.openai.com/docs/guides/function-calling>

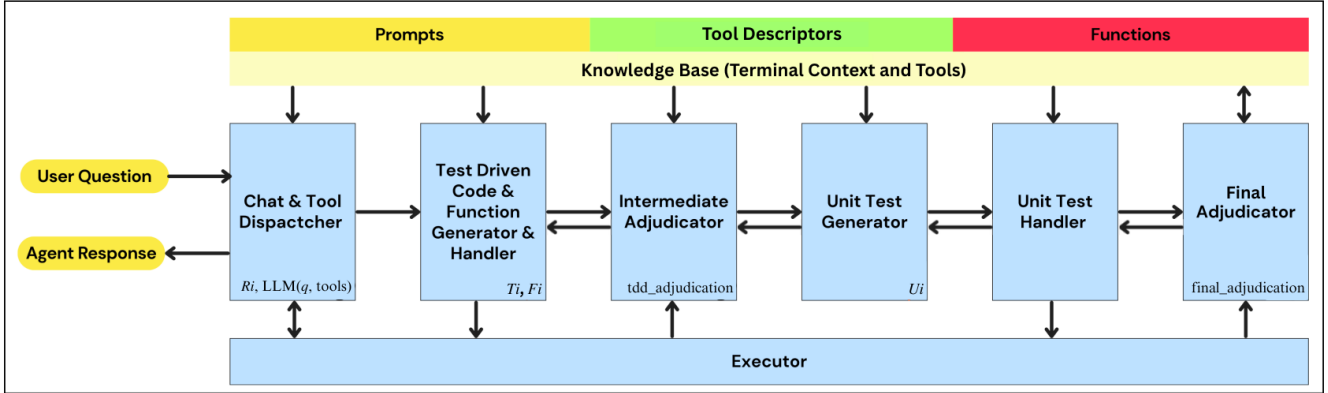


Figure 1: Test-driven generation pipeline. User requirements flow through (1) TDD test generation, (2) function synthesis from tests, (3) intermediate adjudication, and if passed, (4) unit test generation and (5) final adjudication.

system generates code and creates a separate subprocess sandbox for safe execution. Following successful validation, the function is promoted from this isolated sandbox to permanent system integration: added to the Knowledge Base and immediately accessible without restart (through the `importlib.reload()` Python feature), achieving true runtime self-evolution.

2.1 Orchestrated Pipeline Architecture

Figure 1 illustrates the architecture where each component operates through LLM invocations coordinated by deterministic control flow.

The **Knowledge Base** (shown at the top of Figure 1) comprises three persistent repositories. The *Prompts* repository stores system instructions and few-shot examples **of what? specify**. The *Tool Descriptors* repository maintains JSON specifications in OpenAI function-calling format. This enables an external tool or user to request functionalities in natural language, which can then be automatically mapped to precise function calls. The *Functions* repository contains the executable Python implementations. **Terminal Context** maintains persistent subprocess sessions across executions, preserving generated functions and variables between invocations, enabling cross-session capability composition and evolutionary development.

Tools are locally deployed Python functions with associated JSON descriptors. Each descriptor specifies the function signature and parameters, enabling function invocation through OpenAI function-calling. While Functions are the Python code implementations, Tools are functions paired with their JSON descriptors.

The system processes user questions through six sequential components, with the Knowledge Base providing context and the Executor layer ensuring safe execution. The pipeline state at iteration i consists of requirement R_i , test cases T_i , function code F_i , and feedback E_i . We illustrate this process through an actual execution: generating a GCD calculator when a user requests “calculate GCD of 24 and 36.”

Chat & Tool Dispatcher. The entry point in Figure 1 receives user questions and determines if existing functions from the Knowledge Base suffice. For the GCD request, it searches available tools and finds no matching functions:

$$\text{LLM}(q, \text{tools}) \rightarrow \begin{cases} \text{tool_call}(f_j), & \text{if match found} \\ \text{requirement}, & \text{if new needed} \end{cases} \quad (1)$$

Finding no `calculate_gcd` function, The dispatcher produces requirement R_i by employing the LLM to transform the user’s natural language query into a structured function specification for generation.

Test-Driven Code & Function Generator. This dual-purpose component first generates TDD test cases. For GCD, it creates tests like `assert calculate_gcd([24, 36]) == 12`:

$$T_i = \text{LLM}(R_i + E_{i-1}) \quad (2)$$

where E_{i-1} contains reinforcement from previous iterations—accumulated error diagnostics. In the GCD execution, iteration 1’s “Mixed results” feedback became E_0 for iteration 2. The component then synthesizes function code targeting these tests:

$$F_i = \text{LLM}(R_i, T_i) \quad (3)$$

Both operations use specialized prompts from the Knowledge Base to guide generation.

Adjudicators. An adjudicator is an LLM-powered component that acts as an automated judge, evaluating test execution results to make pass/fail decisions about generated code. Our pipeline employs two adjudicators: the Intermediate Adjudicator validates TDD tests, while the Final Adjudicator performs comprehensive evaluation, both determining whether code proceeds or requires regeneration with additional constraints.

Intermediate Adjudicator. This component validates generated code against TDD tests. For GCD, it executes `calculate_gcd([24, 36])` and verifies the result:

$$\text{test_output} = \text{execute_tests}(F_i, T_i) \quad (4)$$

The adjudicator evaluates results through the LLM:

$$\text{tdd_adjudication} = \text{LLM}(\text{test_output}) \rightarrow \begin{cases} \text{accept} \\ \text{reject} + \text{requirements} \end{cases} \quad (5)$$

In the GCD execution, both iterations passed TDD validation, proceeding to comprehensive testing.

Unit Test Generator. After passing intermediate adjudication, the pipeline proceeds to comprehensive testing as shown in Figure 1.

For GCD, it generates edge cases including `calculate_gcd([0, 0])`, negative inputs, and multi-number scenarios:

$$U_i = \text{LLM}(F_i, R_i) \quad (6)$$

This component generates tests for edge cases, error handling, and performance beyond initial TDD specifications.

Unit Test Handler. The handler executes unit tests through the Executor layer, with 30-second timeouts preventing infinite loops. It captures stdout, stderr, exit codes, and returns test results for final adjudication.

Final Adjudicator. The last component in Figure 1’s pipeline evaluates all test results. In the GCD execution, iteration 1 failed with partial test failures, while iteration 2 succeeded:

$$\text{final_adjudication} = \text{LLM}(\text{test_results}) \rightarrow \begin{cases} \text{accept} \\ \text{reject} + \text{requirements} \end{cases} \quad (7)$$

Acceptance adds the function to the Knowledge Base. Rejection triggers the feedback loop with new requirements **what new requirements? please explain.**

System Integration. Upon acceptance, the `calculate_gcd` function permanently integrates into the system through three updates: function code persistence, tool descriptor registration, and prompt enrichment with recognition patterns **these expressions are unclear: what do they mean?**. This integration constitutes the system’s next release—the function becomes a permanent capability immediately available to all future requests. When invoked with [24, 36], it correctly returns 12.

Iterative Refinement. The feedback loops in Figure 1 (back arrows from Final Adjudicator) enable iterative improvement. If the generated function does not pass the tests **explain what happens....**

Each retry incorporates accumulated feedback, progressively constraining the solution space toward convergence.

Safety and Convergence. Dual adjudication ensures safety through progressive validation. Six-iteration limit guarantees termination. Each iteration’s reinforcement constrains the solution space, achieving convergence by treating generated tests as ground truth. The architecture’s isolation mechanisms prevent generated code from compromising core components during testing.

3 Preliminary Evaluation

3.1 Evaluation Methodology

Dataset Selection. We evaluated our approach using HumanEval [2], a standard benchmark containing 164 hand-crafted programming problems with comprehensive test suites. We selected HumanEval over alternatives for specific reasons: MBPP [1] contains simpler problems unsuitable for evaluating complex generation capabilities; CodeContests [14] requires algorithmic competition-level reasoning beyond function synthesis scope; APPS [9] and CodeXGLUE [15] include multi-file projects incompatible with our single-function runtime generation focus. HumanEval provides the optimal balance—challenging enough to demonstrate capability differences, self-contained for runtime execution, and widely adopted enabling direct comparison. Additionally, we used a dataset from the MovieLens 20M dataset³ for testing interface evolution capabilities.

³GroupLens Research MovieLens 20M Dataset accessed via Kaggle. We utilized the `movie.csv` for interface evolution experiments. Available at: <https://www.kaggle.com/datasets/grouplens/movielens-20m-dataset/>

Metrics. Following established practices [2, 5], we measure: (1) *Pass@1* – percentage of problems solved on the first generated solution; (2) *Success Rate* – percentage of generated solutions passing all test cases; (3) *Generation Efficiency* – number of solutions required to achieve success; (4) *Iteration Count* – refinement cycles needed for successful generation.

Statistical Analysis. We applied the Wilcoxon signed-rank test to assess statistical significance of paired comparisons, appropriate for small samples with non-parametric distributions. We report exact p-values and effect sizes ($r = |Z|/n$).

RQ1: Green-field Code Generation. We selected three representative HumanEval problems spanning different complexity levels. For baseline comparison, we chose TiCoder [5] because TiCoder’s dual feedback architecture most closely aligns with our dual adjudicator system. Both employ two-stage validation: TiCoder uses execution results then oracle judgments for progressive filtering, while we use intermediate adjudication then final adjudication for iterative refinement. This architectural parallel—dual feedback mechanisms operating in sequence—makes TiCoder the most appropriate baseline for comparing this aspect of our architecture. We used TiCoder’s oracle-guided evaluation mode where the reference implementation serves as a proxy for user feedback, ensuring unbiased comparison using their own methodology. Both systems were evaluated on: `is_equal_to_sum_even` (logic/arithmetic), `smallest_change` (array manipulation), and `car_race_collision` (mathematical computation). We measured success rate and solution count for direct comparison. We employed GPT-4-0613 for both our framework and TiCoder to ensure fair comparison, as TiCoder’s supported models are limited.

RQ2: Complex Integration Tasks. We designed three complex tasks (Table 1), each evaluated over 5 independent runs. *Salary Analyzer* tests integration with existing employee codebase (597 lines, 4 files, 59 functions, 12 classes) to generate a `salary_analyzer` function that analyzes employee salaries. *Matrix Eigenvalue* tests cross-session composition: generating `matrix_operations`, then creating `advanced_matrix_ops` that builds upon it. *Movie API* tests interface evolution using MovieLens data, requiring multiple return formats with runtime version detection. No baselines exist for runtime self-evolution comparison. We utilized a mixture of GPT-4.1 and o4-mini models to balance generation quality and computational efficiency.

Table 1: Characteristics of long-horizon self-evolution tasks

Task	Integration Type	Description
Salary Analyzer	Existing codebase integration	597 LOC, 12 classes
Matrix Eigenvalue	Cross-session composition	Previous functions
Movie API	Interface evolution	MovieLens dataset

what is context here?

RQ3 Methodology: TDD Components Impact (Ablation Study). We systematically removed architectural components to quantify their contributions. First, we evaluated the complex long-horizon tasks with and without TDD. Second, we evaluated GCD computation with three configurations: full architecture, without TDD, and without both TDD and unit tests. Third, we evaluated 10 HumanEval problems (Table 2) comparing complete architecture against TDD-removed configuration. We employed GPT-4.1 for

GCD computation and a mixture of GPT-4.1 and o4-mini for the long-horizon and HumanEval evaluations.

Table 2: Characteristics of evaluated HumanEval problems

Problem ID	Category (Task)
is_equal_to_sum_even	Logic/Arithmetic (number equals sum and is even)
smallest_change	Array Manipulation (minimum changes for palindrome)
decimal_to_binary	Number Conversion (decimal to binary string)
car_race_collision	Mathematical (calculate collision count)
count_up_to	Prime Generation (generate primes up to n)
split_words	String Processing (split on whitespace/commas)
move_one_ball	Array Rotation (check if sortable by rotation)
match_parens	Bracket Matching (validate parentheses pairing)
rounded_avg	Number Processing (compute rounded average)
sort_array	Sorting Algorithm (sort by binary representation)

3.2 Results and Analysis

RQ1: Green-field Code Generation Capabilities. Table 3 shows our framework achieved 77.8% average success rate compared to TiCoder’s 46.7%, while generating 67% fewer solutions (5 vs 15).

For problems `is_equal_to_sum_even` and `car_race_collision`, our framework achieved success with single generated solutions, while TiCoder required 5 candidate solutions yet achieved only 40% and 20% success rates respectively.

Table 3: Comparative evaluation on HumanEval problems

Problem	Our Framework	TiCoder
is_equal_to_sum_even	1/1 solutions (100%)	2/5 solutions (40%)
smallest_change	1/3 solutions (33%)	4/5 solutions (80%)
car_race_collision	1/1 solutions (100%)	1/5 solutions (20%)
Average	77.8%	46.7%
Total	3/5 solutions	7/15 solutions

Wilcoxon test: $W=1.0$, $p=0.50$, $r=0.62$ (large effect)

RQ2: Self-Evolution Through Complex Long-Horizon Tasks.

Table 4 shows results from 5 runs per task. Matrix Eigenvalue (3/5 success), Salary Analyzer (2/5 success), and Movie API (2/5 success) demonstrate mixed performance.

Table 4: Self-evolution tasks (5 runs each)

Task	Success Rate
Matrix Eigenvalue	3/5 (60%)
Salary Analyzer	2/5 (40%)
Movie API	2/5 (40%)

Matrix Eigenvalue achieved 60% success rate through capability composition, while Salary Analyzer and Movie API showed 40% success rates, revealing challenges in codebase integration and interface versioning.

RQ3: Architectural Component Impact (Ablation Study).

We conducted comprehensive ablation studies to isolate component contributions. Table 5 shows TDD’s critical role in complex long-horizon tasks:

Across 30 total experiments (15 with TDD, 15 without), TDD improved performance on Matrix Eigenvalue (60% vs 0%) and Salary Analyzer (40% vs 20%), though Movie API performed worse (40% vs 60%). While not statistically significant ($p=0.75$), the medium effect size ($r=0.46$) suggests practical importance.

Table 6 shows progressive capability enhancement through architectural layers, with TDD integration yielding 50% iteration reduction while maintaining perfect accuracy.

Table 5: TDD impact on self-evolution (5 runs per task)

Task	With TDD	Without TDD
Matrix Eigenvalue	3/5 (60%)	0/5 (0%)
Salary Analyzer	2/5 (40%)	1/5 (20%)
Movie API	2/5 (40%)	3/5 (60%)

Wilcoxon test: $W=1.5$, $p=0.75$, $r=0.46$ (medium effect)

Table 6: Layer ablation study on GCD computation

Architecture	Iterations	Improvement
Framework w/o Unit Test and TDD	4	Baseline
Framework w/o TDD	3	25% reduction
Framework (Our Approach)	2	50% reduction

Table 7 validates component criticality on 10 HumanEval problems:

Table 7: Direct comparison on 10 HumanEval problems

Configuration	Pass@1	Problems Solved
Complete Architecture	100%	10/10
Without TDD Component	90%	9/10

Wilcoxon test: $W=0$, $p=1.00$ (single discordant pair)

The complete architecture achieved perfect Pass@1 of 100%, while removing TDD reduced Pass@1 to 90%, failing on `move_one_ball`. Though the single discordant pair prevented statistical significance ($p=1.00$), the practical impact is clear: TDD eliminated all failures.

4 Future Work and Conclusion

Future work will enrich the framework with change impact analysis to trace modification consequences, requirements negotiation protocols to resolve compatibility conflicts that occurred during our evaluation, and regression testing to ensure safe evolution, alongside evaluation on larger datasets for adequate statistical significance.

Implications. This paradigm enables unprecedented development cycles—functions generated in seconds, not weeks—with immediate user feedback loops transforming how software adapts to needs. For research, it challenges fundamental assumptions about software mutability and opens new directions in self-modifying systems. For practice, it promises systems that evolve with user needs in real-time, reducing development bottlenecks and enabling instant feature deployment.

Threats to Validity. *Internal:* Limited evaluation to function-level generation may not reveal system-level complexities. *External:* Results may not generalize beyond single-function evolution. *Construct:* Pass@1 metrics may not capture code quality, maintainability, or security implications. *Statistical:* Small sample sizes ($n=3$) limited statistical power; while effect sizes were large ($r=0.62$), significance was not achieved ($p0.50$). *Reliability:* Framework depends on LLM capabilities which vary across models.

We demonstrated that software self-evolution through runtime code generation is achievable with appropriate architectural safeguards. Despite limited statistical power, large effect sizes ($r=0.46$ - 0.62) and consistent performance improvements suggest our approach’s practical value for autonomous capability extension.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). Introduces the Mostly Basic Python Problems (MBPP) dataset.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [3] Chrysanthos Dellarocas, Mark Klein, and Howard Shrobe. 1998. An architecture for constructing self-evolving software systems. In *Proceedings of the third international workshop on Software architecture*. 29–32.
- [4] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A survey on code generation with llm-based agents. *arXiv preprint arXiv:2508.00083* (2025).
- [5] Sarah Fakhoury, Saikat Chakraborty, Madanlal Musuvathi, and Shuvendu K. Lahiri. 2024. Test-driven interactive code generation. In *Proceedings of the 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, USA.
- [6] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. *arXiv:2508.07407 [cs.AI]* <https://arxiv.org/abs/2508.07407>
- [7] GitHub. 2021. GitHub Copilot: Your AI pair programmer. <https://github.com/features/copilot>
- [8] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [9] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. *arXiv preprint arXiv:2105.09938* (2021).
- [10] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Xiong, Yuheng Fu, Zili Cheng, Shengyi Zhang, Jing Wang, Jinlin Zheng, Shuyang Li, et al. 2023. MetaGPT: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023).
- [11] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [12] Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. 2025. ReVeal: Self-Evolving Code Agents via Iterative Generation-Verification. *arXiv preprint arXiv:2506.11442* (2025).
- [13] Neil D Jones, Carsten K Gomard, and Peter Sestoft. 1993. *Partial evaluation and automatic program generation*. Peter Sestoft.
- [14] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. CodeContests: Competitive programming with AlphaCode. In *Science*, Vol. 378. 1092–1097.
- [15] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).
- [16] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqian Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2332–2354.
- [17] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2025. How much does AI impact development speed? An enterprise-based randomized controlled trial. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 618–629.
- [18] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 8696–8708.
- [19] Danny Weyns, Thomas Bäck, Rene Vidal, Xin Yao, and Ahmed Nabil Belbachir. 2023. The vision of self-evolving computing systems. *Journal of Integrated Design and Process Science* 26, 3–4 (2023), 351–367.
- [20] Xingyu Wu, Sheng-hao Wu, Jibin Wu, Liang Feng, and Kay Chen Tan. 2024. Evolutionary computation in the era of large language model: Survey and roadmap. *IEEE Transactions on Evolutionary Computation* (2024).